

03 — LLVM, LLVM IR, and llvmlite

What LLVM Is

LLVM (Low Level Virtual Machine — the name is now a legacy misnomer; it's a compiler infrastructure, not a VM) is a collection of reusable compiler components developed at the University of Illinois and now used by Apple (Swift, Objective-C), Mozilla (Rust), Google (TensorFlow XLA), and many others.

The core value LLVM provides to a language implementer: **you describe your program in LLVM IR, and LLVM turns it into efficient native machine code for any CPU architecture (x86, ARM, RISC-V, WebAssembly, etc.) automatically.**

Without LLVM, building a native compiler means implementing:

- Instruction selection (which CPU instruction to use for `x + y`?)
- Register allocation (which of the 16 available CPU registers holds which value?)
- Calling conventions (how does `func(a, b)` actually pass `a` and `b` to the function in machine code?)
- Peephole optimisations, loop unrolling, inlining, constant folding...

LLVM has implemented all of this for every major architecture over 20+ years. JOCKY uses it for free.

The Two LLVM Layers in llvmlite

llvmlite exposes LLVM through two Python modules:

`llvmlite.ir` — the construction layer

Pure Python. No LLVM binary is involved. This module lets you *describe* your program using Python objects:

```
import llvmlite.ir as ir

module = ir.Module(name='hello')
i64     = ir.IntType(64)
fn_type = ir.FunctionType(i64, [i64, i64])
fn      = ir.Function(module, fn_type, name='add')
block   = fn.append_basic_block('entry')
builder = ir.IRBuilder(block)
result  = builder.add(fn.args[0], fn.args[1], 'sum')
builder.ret(result)
```

`str(module)` converts the Python objects to LLVM IR text:

```
define i64 @add(i64 %0, i64 %1) {
entry:
    %sum = add i64 %0, %1
```

```
    ret i64 %sum
}
```

Nothing is compiled yet. The IR layer just builds a description.

llvmlite.binding — the engine layer

Wraps the actual LLVM C++ library (compiled to `llvmlite.dll/llvmlite.so`). Receives the IR text string and invokes LLVM to:

- Parse and validate the IR
- Optimise it
- Compile it to machine code for the current platform
- (In JIT mode) execute it in-process

```
import llvmlite.binding as llvm

llvm.initialize_native_target()
llvm.initialize_native_asmprinter()

ir_text = str(module)          # get IR text from the construction layer
llvm_mod = llvm.parse_assembly(ir_text) # hand to LLVM's parser
llvm_mod.verify()              # validate IR is well-formed

target = llvm.Target.from_default_triple()
tm      = target.create_target_machine(opt=2)
obj_data = tm.emit_object(llvm_mod) # compile to machine code bytes
```

LLVM IR — The Format

LLVM IR is a typed, SSA-based, human-readable (also machine-parseable) representation of programs. Here is the anatomy:

Module

The top-level container. One module per source file.

```
; Module ID = 'hello'
source_filename = "hello"
target datalayout = "e-m:w-p270:32:32-..."
target triple = "x86_64-pc-windows-gnu"
```

`target triple` tells LLVM what platform to compile for. `from_default_triple()` sets this to the current machine's platform automatically.

External declarations

Functions that the module *uses* but does not *define*. The linker (or JIT resolver) provides them.

```
declare void @report(i8*)
declare i64 @proc_count(i8*)
declare i32 @strcmp(i8*, i8*)
```

@ prefix means it's a global name. `i8*` is a pointer to 8-bit integer (C `char*`).

Function definition

```
define void @start() {
entry:
; instructions here
ret void
}
```

`define` means this module contains the implementation. The block named `entry` is the function's first basic block.

Global variables

```
@".jk_str.0" = internal constant [19 x i8] c"Hello from JOCKY!\00"
```

- `internal` — not visible outside this module (like `static` in C)
- `constant` — the value never changes; goes in read-only section
- `[19 x i8]` — an array of 19 8-bit integers
- `c"..."` — a C-style string literal initialiser

LLVM Types Used by JOCKY

LLVM Type	Bits	What it represents	JOCKY type
<code>i1</code>	1	Boolean (0 or 1)	<code>flag</code>
<code>i8</code>	8	Byte	(internal — strings)
<code>i32</code>	32	32-bit integer	(internal — strcmp result, GEP indices)
<code>i64</code>	64	64-bit signed integer	<code>num</code>
<code>double</code>	64	IEEE 754 double-precision float	<code>dec</code>
<code>i8*</code>	64 (pointer)	Pointer to bytes (C <code>char*</code> or <code>void*</code>)	<code>text</code> , <code>raw</code>
<code>void</code>	—	No value	<code>nothing</code>
<code>[N x i8]</code>	N×8	Array of N bytes	(string storage)

llvmlite constructors:

```
i1    = ir.IntType(1)
i8    = ir.IntType(8)
i32   = ir.IntType(32)
i64   = ir.IntType(64)
f64   = ir.DoubleType()
void  = ir.VoidType()
i8p   = ir.IntType(8).as_pointer()
arr_t = ir.ArrayType(i8, 19) # [19 x i8]
```

SSA Form — Static Single Assignment

A fundamental constraint of LLVM IR: **every value is assigned exactly once and never changes.**

```
%x = add i64 5, 3      ; OK: x is assigned once
%x = add i64 %x, 1     ; ILLEGAL: x is assigned again
```

This is not a limitation — it is a feature. SSA form enables LLVM to reason about values without any aliasing analysis. Optimisations become simpler and more powerful.

How variables work in SSA

JOCKY variables are mutable. How do we represent mutation in SSA? By keeping the variable's value in *memory* (on the stack), not in an SSA register.

```
var x : num := 5
x <- x + 1
```

becomes:

```
%x = alloca i64          ; create a stack slot for x
store i64 5, i64* %x     ; write 5 into the slot
%x.0 = load i64, i64* %x ; read current value
%x.1 = add i64 %x.0, 1    ; compute x + 1
store i64 %x.1, i64* %x  ; write new value back
```

- `%x` (the `alloca` pointer) is assigned once and never changes — it always points to the same stack slot.
- The *contents* of the slot change via `store`, but the pointer itself is SSA.
- `load` always reads the current contents.

LLVM's `mem2reg` pass (one of the first optimisations applied) detects these `alloca/load/store` patterns and promotes them to proper SSA PHI nodes when possible, eliminating the memory accesses in the final binary.

Basic Blocks and Control Flow

A **basic block** is a sequence of instructions with:

- Exactly one entry point (the first instruction)
- Exactly one exit point (the last instruction, which must be a *terminator*)

Terminators are:

- `ret` — return from the function
- `br label %target` — unconditional branch to another block
- `cbranch i1 %cond, label %true, label %false` — conditional branch

Every basic block's last instruction must be one of these. The IR verifier (`llvm_mod.verify()`) checks this. If you try to emit an instruction after a terminator, `llvmlite` will raise an error.

Control flow graph

The blocks form a directed graph (CFG — Control Flow Graph). LLVM analyses this graph for optimisations and correctness checks.

```
For:  check (x gt 5) { report(`big`) } otherwise { report(`small`) }
```

CFG:



The IRBuilder

The `ir.IRBuilder` is your tool for emitting instructions. It tracks the *current insertion point* — which basic block, at which position, new instructions are added.

```

builder = ir.IRBuilder(entry_block)  # start at the end of entry_block

a = builder.alloca(i64, name='a')     # emits: %a = alloca i64
builder.store(ir.Constant(i64, 5), a) # emits: store i64 5, i64* %a

# Move to another block:
builder.position_at_end(then_block)

# Check if current block already has a terminator:
if not builder.block.is_terminated:
    builder.branch(merge_block)
  
```

Key builder methods used in JOCKY:

Method	IR emitted	Purpose
<code>builder.alloca(type, name)</code>	<code>%name = alloca type</code>	Allocate stack variable
<code>builder.store(val, ptr)</code>	<code>store type val, type* ptr</code>	Write to stack slot
<code>builder.load(ptr, name)</code>	<code>%name = load type, type* ptr</code>	Read from stack slot
<code>builder.add(a, b, name)</code>	<code>%name = add i64 a, b</code>	Integer addition
<code>builder.sub(a, b, name)</code>	<code>%name = sub i64 a, b</code>	Integer subtraction
<code>builder.mul(a, b, name)</code>	<code>%name = mul i64 a, b</code>	Integer multiplication
<code>builder.sdiv(a, b, name)</code>	<code>%name = sdiv i64 a, b</code>	Signed integer division
<code>builder.srem(a, b, name)</code>	<code>%name = srem i64 a, b</code>	Signed remainder (modulo)
<code>builder.fadd(a, b, name)</code>	<code>%name = fadd double a, b</code>	Float addition
<code>builder.fsub(a, b, name)</code>	<code>%name = fsub double a, b</code>	Float subtraction
<code>builder.fmul(a, b, name)</code>	<code>%name = fmul double a, b</code>	Float multiplication
<code>builder.fdiv(a, b, name)</code>	<code>%name = fdiv double a, b</code>	Float division
<code>builder.icmp_signed(op, a, b, name)</code>	<code>%name = icmp sgt i64 a, b</code>	Signed integer compare
<code>builder.fcmp_ordered(op, a, b, name)</code>	<code>%name = fcmp oeq double a, b</code>	Ordered float compare
<code>builder.and_(a, b, name)</code>	<code>%name = and i1 a, b</code>	Bitwise AND (used for also)
<code>builder.or_(a, b, name)</code>	<code>%name = or i1 a, b</code>	Bitwise OR (used for or)
<code>builder.xor(a, b, name)</code>	<code>%name = xor i1 a, 1</code>	Bitwise XOR (used for flip)
<code>builder.not_(a, name)</code>	<code>%name = xor i1 a, true</code>	Logical NOT
<code>builder.sitofp(val, f64, name)</code>	<code>%name = sitofp i64 val to double</code>	Int to float
<code>builder.fptosi(val, i64, name)</code>	<code>%name = fptosi double val to i64</code>	Float to int
<code>builder.zext(val, i64, name)</code>	<code>%name = zext i1 val to i64</code>	Zero-extend
<code>builder.trunc(val, i1, name)</code>	<code>%name = trunc i64 val to i1</code>	Truncate
<code>builder.gep(ptr, idxs, inbounds)</code>	<code>%p = getelementptr inbounds ...</code>	Pointer arithmetic

Method	IR emitted	Purpose
<code>builder.call(fn, args, name)</code>	<code>%name = call void @fn(...)</code>	Function call
<code>builder.ret(val)</code>	<code>ret i64 val</code>	Return with value
<code>builder.ret_void()</code>	<code>ret void</code>	Return without value
<code>builder.branch(block)</code>	<code>br label %block</code>	Unconditional jump
<code>builder.cbranch(cond, t, f)</code>	<code>br i1 cond, label %t, label %f</code>	Conditional jump

GEP — Get Element Pointer

GEP is one of LLVM's most important (and confusing) instructions. It computes a pointer to an element inside an aggregate (array or struct) without loading any data.

For JOCKY's string globals:

```
@".jk_str.0" = internal constant [19 x i8] c"Hello from JOCKY!\00"
```

The type of `@".jk_str.0"` is `[19 x i8]*` (pointer to array of 19 bytes). We need an `i8*` (pointer to the first byte). GEP computes this:

```
%strptr = getelementptr inbounds [19 x i8], [19 x i8]* @".jk_str.0", i32 0, i32 0
```

Read as:

- Starting from the pointer `@".jk_str.0"`
- Index `0` into the outer type (select the first — and only — array)
- Index `0` into the array (select the first element)
- Result: `i8*` pointing to the first byte

The two indices `[0, 0]` are because the global is a *pointer to an array*, not the array itself. The first `0` dereferences the pointer-to-array level; the second `0` indexes into the array.

This is a pure address computation — no memory access happens. `inbounds` tells LLVM the result is within the allocation bounds, enabling certain optimisations.

JIT Compilation — How It Works

JIT (Just-In-Time) compilation compiles code at program runtime instead of ahead-of-time.

JOCKY's JIT flow:

```

# 1. Build the IR module using llvmlite.ir (pure Python)
ir_module = CodeGenerator(...).generate(ast)

# 2. Convert to IR text
ir_text = str(ir_module)

# 3. Parse the text in the binding layer (invokes LLVM C++ library)
llvm_mod = llvm.parse_assembly(ir_text)
llvm_mod.verify()

# 4. Create a target machine for the current CPU
target = llvm.Target.from_default_triple()
tm      = target.create_target_machine()

# 5. Create an MCJIT execution engine
with llvm.create_mcjit_compiler(llvm_mod, tm) as engine:

    # 6. Register external functions BEFORE linking
    #     (LLVM resolves @report, @procs_list, etc. to these addresses)
    register_all()

    # 7. Compile and link (in-process)
    engine.finalize_object()
    engine.run_static_constructors()

    # 8. Get the address of the 'start' function in compiled machine code
    func_addr = engine.get_function_address('start')

    # 9. Create a Python-callable wrapper
    start_fn = ctypes.CFUNCTYPE(None)(func_addr)

    # 10. Call it – pure native machine code runs
    start_fn()

```

MCJIT (Machine Code JIT) is the JIT engine variant used. It compiles entire modules at once (rather than function-by-function like the older JIT). Once `finalize_object()` is called, all functions are compiled to native machine code and available at their addresses.

Symbol resolution: When JOCKY's compiled code calls `@report(...)`, LLVM's linker needs to find the actual memory address of `report`. `register_all()` calls `llvm.add_symbol('report', address)` for each stdlib function, where `address` is the memory address of a Python ctypes callback. LLVM resolves the call to that address.

AOT Compilation — Object Files

AOT (Ahead-of-Time) compilation, the default mode, produces a `.o` object file:

```

target = llvm.Target.from_default_triple()
tm      = target.create_target_machine(opt=2, reloc='static')

```



```
obj_bytes = tm.emit_object(llvm_mod)
with open('hello.o', 'wb') as f:
    f.write(obj_bytes)
```

An object file is machine code that has NOT been linked yet. It contains:

- **Code section (.text):** The compiled function instructions
- **Data section (.data/.rdata):** Global variables and constants (string literals)
- **Symbol table:** Names of functions and globals defined or referenced
- **Relocation table:** Locations that need to be patched by the linker

The relocation table is why we need the linker. When the compiled `start()` function calls `report()`, it emits a `call` instruction with a placeholder address. The linker replaces that placeholder with the actual address of `report` from `forensics.o`.

reloc='static': On Windows with MinGW, the linker expects Position-Dependent Code (PDC) for executables. The default LLVM relocation model produces Position-Independent Code (PIC) which uses a Global Offset Table (GOT) — a mechanism for shared libraries. MinGW's exe linker doesn't set up a GOT for executables, causing linker errors (`undefined reference to _GLOBAL_OFFSET_TABLE_`). Specifying **reloc='static'** tells LLVM to emit code that doesn't require a GOT.

opt=2: Runs LLVM's `-O2` optimisation pipeline: constant folding, dead code elimination, loop optimisation, inlining, etc. Makes the final binary faster and smaller.